```
# STEP 0A — Install BioPython and MAFFT

!pip install biopython -q
import os, sys, Bio
print("BioPython version:", Bio.__version__)

!apt-get install -qq -y mafft
!mafft --version
```

```
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 3.2/3.2 MB 10.6 MB/s eta 0:00:00
BioPython version: 1.87
Extracting templates from packages: 100%
Selecting previously unselected package fonts-lato.
(Reading database ... 118194 files and directories currently installed.)
Preparing to unpack .../00-fonts-lato_2.0-2.1_all.deb ...
Unpacking fonts-lato (2.0-2.1) ...
Selecting previously unselected package netbase.
Preparing to unpack .../01-netbase_6.3_all.deb ...
Unpacking netbase (6.3) ...
Selecting previously unselected package libclone-perl.
Preparing to unpack .../02-libclone-perl_0.45-1build3_amd64.deb ...
Unpacking libclone-perl (0.45-1build3) ...
Selecting previously unselected package libdata-dump-perl.
Preparing to unpack .../03-libdata-dump-perl_1.25-1_all.deb ...
Unpacking libdata-dump-perl (1.25-1) ...
Selecting previously unselected package libencode-locale-perl.
Preparing to unpack .../04-libencode-locale-perl_1.05-1.1_all.deb ...
Unpacking libencode-locale-perl (1.05-1.1) ...
Selecting previously unselected package libhttp-date-perl.
Preparing to unpack .../05-libhttp-date-perl_6.05-1_all.deb ...
Unpacking libhttp-date-perl (6.05-1) ...
Selecting previously unselected package libfile-listing-perl.
Preparing to unpack .../06-libfile-listing-perl_6.14-1_all.deb ...
Unpacking libfile-listing-perl (6.14-1) ...
Selecting previously unselected package libfont-afm-perl.
Preparing to unpack .../07-libfont-afm-perl_1.20-3_all.deb ...
Unpacking libfont-afm-perl (1.20-3) ...
Selecting previously unselected package libhtml-tagset-perl.
Preparing to unpack .../08-libhtml-tagset-perl_3.20-4_all.deb ...
Unpacking libhtml-tagset-perl (3.20-4) ...
Selecting previously unselected package liburi-perl.
Preparing to unpack .../09-liburi-perl_5.10-1_all.deb ...
Unpacking liburi-perl (5.10-1) ...
Selecting previously unselected package libhtml-parser-perl:amd64.
Preparing to unpack .../10-libhtml-parser-perl_3.76-1build2_amd64.deb ...
Unpacking libhtml-parser-perl:amd64 (3.76-1build2) ...
Selecting previously unselected package libio-html-perl.
Preparing to unpack .../11-libio-html-perl_1.004-2_all.deb ...
Unpacking libio-html-perl (1.004-2) ...
Selecting previously unselected package liblwp-mediatypes-perl.
Preparing to unpack .../12-liblwp-mediatypes-perl_6.04-1_all.deb ...
Unpacking liblwp-mediatypes-perl (6.04-1) ...
Selecting previously unselected package libhttp-message-perl.
Preparing to unpack .../13-libhttp-message-perl_6.36-1_all.deb ...
Unpacking libhttp-message-perl (6.36-1) ...
Selecting previously unselected package libhtml-form-perl.
Preparing to unpack .../14-libhtml-form-perl_6.07-1_all.deb ...
Unpacking libhtml-form-perl (6.07-1) ...
Selecting previously unselected package libhtml-tree-perl.
Preparing to unpack .../15-libhtml-tree-perl_5.07-2_all.deb ...
Unpacking libhtml-tree-perl (5.07-2) ...
Selecting previously unselected package libhtml-format-perl.
Preparing to unpack .../16-libhtml-format-perl_2.12-1.1_all.deb ...
Unpacking libhtml-format-perl (2.12-1.1) ...
Selecting previously unselected package libhttp-cookies-perl.
Preparing to unpack .../17-libhttp-cookies-perl_6.10-1_all.deb ...
Unpacking libhttp-cookies-perl (6.10-1) ...
```

```
# STEP 0B — Install ModelTest-NG and RAxML-NG

!pip install -q condacolab
import condacolab
condacolab.install_miniconda()
```

```
Miniconda is subject to terms of service: https://legal.anaconda.com/policies/en/#terms-of-service
⏬ Downloading https://repo.anaconda.com/miniconda/Miniconda3-py311_24.11.1-0-Linux-x86_64.sh...
---------------------------------------------------------------------------
AssertionError                            Traceback (most recent call last)
/tmp/ipykernel_7364/1405566202.py in <cell line: 0>()
      3 get_ipython().system('pip install -q condacolab')
      4 import condacolab
----> 5 condacolab.install_miniconda()

                                    1 frames
/usr/local/lib/python3.12/dist-packages/condacolab.py in install_from_url(installer_url, prefix, env, run_checks, sha256)
     97         digest = _chunked_sha256(installer_fn)
     98         assert (
---> 99             digest == sha256
    100         ), f"💥 💔 💥 Checksum failed! Expected {sha256}, got {digest}"
    101

AssertionError: 💥 💔 💥 Checksum failed! Expected 35a58b8961e1187e7311b979968662c6223e86e1451191bed2e67a72b6bd0658, got
807774bae6cd87132094458217ebf713df436f64779faf9bb4c3d4b6615c1e3a
```

```
# STEP 0B — Install ModelTest-NG and RAxML-NG (no condacolab needed)

# Install RAxML-NG binary directly
!wget -q https://github.com/amkozlov/raxml-ng/releases/download/1.2.0/raxml-ng_v1.2.0_linux_x86_64.zip
!unzip -q raxml-ng_v1.2.0_linux_x86_64.zip
!cp raxml-ng /usr/local/bin/raxml-ng
!chmod +x /usr/local/bin/raxml-ng

# Install ModelTest-NG binary directly
!wget -q https://github.com/ddarriba/modeltest/releases/download/v0.1.7/modeltest-ng-0.1.7-linux-x86_64.tar.gz
!tar -xzf modeltest-ng-0.1.7-linux-x86_64.tar.gz
!cp modeltest-ng-0.1.7-linux-x86_64/bin/modeltest-ng /usr/local/bin/modeltest-ng
!chmod +x /usr/local/bin/modeltest-ng

print("Done. Verifying...")
!raxml-ng --version
!modeltest-ng --version
```

```
tar (child): modeltest-ng-0.1.7-linux-x86_64.tar.gz: Cannot open: No such file or directory
tar (child): Error is not recoverable: exiting now
tar: Child returned status 2
tar: Error is not recoverable: exiting now
cp: cannot stat 'modeltest-ng-0.1.7-linux-x86_64/bin/modeltest-ng': No such file or directory
chmod: cannot access '/usr/local/bin/modeltest-ng': No such file or directory
Done. Verifying...

RAxML-NG v. 1.2.0 released on 09.05.2023 by The Exelixis Lab.
Developed by: Alexey M. Kozlov and Alexandros Stamatakis.
Contributors: Diego Darriba, Tomas Flouri, Benoit Morel, Sarah Lutteropp, Ben Bettisworth, Julia Haag, Anastasis Togkousidis.
Latest version: https://github.com/amkozlov/raxml-ng
Questions/problems/suggestions? Please visit: https://groups.google.com/forum/#!forum/raxml

System: Intel(R) Xeon(R) CPU @ 2.20GHz, 1 cores, 12 GB RAM

/bin/bash: line 1: modeltest-ng: command not found
```

```
# STEP 0C — Verify Installation

!which modeltest-ng && modeltest-ng --version
!which raxml-ng    && raxml-ng --version
```

```
/usr/local/bin/raxml-ng

RAxML-NG v. 1.2.0 released on 09.05.2023 by The Exelixis Lab.
Developed by: Alexey M. Kozlov and Alexandros Stamatakis.
Contributors: Diego Darriba, Tomas Flouri, Benoit Morel, Sarah Lutteropp, Ben Bettisworth, Julia Haag, Anastasis Togkousidis.
Latest version: https://github.com/amkozlov/raxml-ng
Questions/problems/suggestions? Please visit: https://groups.google.com/forum/#!forum/raxml

System: Intel(R) Xeon(R) CPU @ 2.20GHz, 1 cores, 12 GB RAM
```

```
# STEP 1 — Retrieve BOTH Query Sequences from NCBI
# Query A: Human KIF5B (NP_004512) — Kinesin motor (Kinesin-1), N-terminal, plus-end directed
# Query B: Human KIFC1 (NP_002254) — Kinesin-14 (HSET), C-terminal, minus-end directed

from Bio import SeqIO, Entrez
```

```
import time, os

Entrez.email = "your.email@example.com"   # <-- Replace with your email

queries = {
    "KIF5B_Hs_Kinesin1":  "NP_004512",   # Human KIF5B — conventional kinesin motor (Kinesin-1)
    "KIFC1_Hs_Kinesin14": "NP_002254",   # Human KIFC1 — Kinesin-14 (HSET)
}

os.makedirs("data", exist_ok=True)
query_records = {}

for name, acc in queries.items():
    handle = Entrez.efetch(db="protein", rettype="fasta", id=acc)
    rec = SeqIO.read(handle, "fasta")
    handle.close()
    rec.id = name
    rec.description = ""
    query_records[name] = rec
    fasta_path = f"data/{name}.fasta"
    SeqIO.write(rec, fasta_path, "fasta")
    print(f"[{name}]  length={len(rec.seq)}  saved → {fasta_path}")
    time.sleep(0.5)
```

```
[KIF5B_Hs_Kinesin1]  length=963  saved → data/KIF5B_Hs_Kinesin1.fasta
[KIFC1_Hs_Kinesin14]  length=673  saved → data/KIFC1_Hs_Kinesin14.fasta
```

```
# STEP 2A — BLAST Search 1: Kinesin Motor (KIF5B)
# BLASTp against PDB, no organism filter
# Retrieves diverse cross-species kinesin motor hits
# NOTE: This may take 3-5 minutes

%%time
from Bio.Blast import NCBIWWW

result_kinesin = NCBIWWW.qblast(
    "blastp",
    "pdb",
    query_records["KIF5B_Hs_Kinesin1"].seq,
    hitlist_size=30
)
with open("data/blast_kinesin_motor.xml", "w") as f:
    f.write(result_kinesin.read())
result_kinesin.close()
print("Done → data/blast_kinesin_motor.xml")
```

```
Done → data/blast_kinesin_motor.xml
CPU times: user 21.6 ms, sys: 10.1 ms, total: 31.7 ms
Wall time: 1min 2s
```

```
# STEP 2B — BLAST Search 2: Kinesin-14 (KIFC1/HSET)
# BLASTp against PDB, no organism filter
# Retrieves Ncd, Kar3, Pkl1, XCTK2, klpA and other Kinesin-14 members
# NOTE: This may take 3-5 minutes

%%time
from Bio.Blast import NCBIWWW

result_k14 = NCBIWWW.qblast(
    "blastp",
    "pdb",
    query_records["KIFC1_Hs_Kinesin14"].seq,
    hitlist_size=30
)
with open("data/blast_kinesin14.xml", "w") as f:
    f.write(result_k14.read())
result_k14.close()
print("Done → data/blast_kinesin14.xml")
```

```
Done → data/blast_kinesin14.xml
CPU times: user 21.3 ms, sys: 3.88 ms, total: 25.2 ms
Wall time: 1min 2s
```

```
# STEP 3 — Parse Both BLAST Results into structured objects using NCBIXML

from Bio.Blast import NCBIXML
```

```
with open("data/blast_kinesin_motor.xml") as f:
    blast_kinesin = NCBIXML.read(f)

with open("data/blast_kinesin14.xml") as f:
    blast_k14 = NCBIXML.read(f)

print(f"Kinesin motor BLAST hits:  {len(blast_kinesin.alignments)}")
print(f"Kinesin-14 BLAST hits:     {len(blast_k14.alignments)}")
```

```
Kinesin motor BLAST hits:  30
Kinesin-14 BLAST hits:     30
```

```
# STEP 4 — View BLAST Hits for Both Searches
# Prints Hit ID, alignment length, E-value, and percent identity for each hit

def print_hits(blast_record, label):
    print(f"\n{label}")
    print(f"{'Hit ID':<28} {'Aln Len':>8} {'E-value':>12} {'PID%':>7}")
    print("-" * 60)
    for aln in blast_record.alignments:
        for hsp in aln.hsps:
            pid = 100 * hsp.identities / hsp.align_length
            print(f"{aln.hit_id:<28} {hsp.align_length:>8} {hsp.expect:>12.2e} {pid:>7.1f}")

print_hits(blast_kinesin, "KINESIN MOTOR (KIF5B) hits")
print_hits(blast_k14,    "KINESIN-14 (KIFC1) hits")
```

```
KINESIN MOTOR (KIF5B) hits
Hit ID                       Aln Len      E-value    PID%
------------------------------------------------------------
pdb|8RHB|K                       963     0.00e+00   100.0
pdb|9GNQ|K                       357     0.00e+00   100.0
pdb|9GNQ|K                        24     1.14e-02    70.8
pdb|1MKJ|A                       349     0.00e+00   100.0
pdb|8IXA|S                       349     0.00e+00    99.7
pdb|9L7M|K                       349     0.00e+00    99.4
pdb|3J8X|K                       349     0.00e+00    98.9
pdb|4HNA|K                       349     0.00e+00    98.3
pdb|9L7E|A                       349     0.00e+00    98.0
pdb|4ATX|C                       340     0.00e+00    98.8
pdb|1BG2|A                       325     0.00e+00   100.0
pdb|9L78|A                       335     0.00e+00    97.9
pdb|9L6K|A                       335     0.00e+00    97.9
pdb|4LNU|K                       325     0.00e+00    98.5
pdb|5LT3|A                       325     0.00e+00    98.2
pdb|5LT1|A                       325     0.00e+00    98.2
pdb|6OJQ|K                       317     0.00e+00    98.7
pdb|3J6H|K                       342     0.00e+00    86.8
pdb|3WRD|A                       334     0.00e+00    87.1
pdb|4UXT|C                       334     0.00e+00    85.3
pdb|5HNY|K                       321     0.00e+00    87.2
pdb|5HLE|A                       321     0.00e+00    85.7
pdb|2Y5W|A                       355     0.00e+00    75.8
pdb|2KIN|A                       238    3.86e-148    85.3
pdb|1GOJ|A                       347    8.08e-126    55.6
pdb|1IA0|K                       372     3.90e-84    42.5
pdb|3B6U|A                       342     2.60e-80    45.3
pdb|1I5S|A                       363     3.70e-79    42.1
pdb|1VFV|A                       363     4.69e-79    42.1
pdb|1I6I|A                       363     4.93e-79    42.1
pdb|7A3Z|A                       343     8.63e-79    45.2

KINESIN-14 (KIFC1) hits
Hit ID                       Aln Len      E-value    PID%
------------------------------------------------------------
pdb|5WDH|A                       357     0.00e+00    99.7
pdb|2NCD|A                       433     5.49e-97    41.8
pdb|4GKR|A                       378     1.18e-95    44.7
pdb|5W3D|A                       421     1.91e-95    42.5
pdb|1CZ7|A                       413     7.21e-95    42.4
pdb|3U06|A                       421     1.54e-94    42.3
pdb|1N6M|A                       421     1.81e-94    42.3
pdb|8YUE|A                       421     3.55e-94    42.3
pdb|8YY2|C                       421     5.49e-94    42.3
pdb|4ETP|A                       412     6.35e-94    41.5
pdb|1F9T|A                       379     7.30e-94    44.6
pdb|3L1C|A                       412     1.92e-93    42.5
pdb|3T0Q|A                       371     1.23e-90    45.8
pdb|3KAR|A                       367     2.16e-86    43.6
```

```
pdb|1F9W|A                        369     5.62e-86    43.1
pdb|1F9U|A                        369     6.52e-86    43.1
pdb|1F9V|A                        369     7.57e-86    43.1
pdb|4H1G|A                        364     1.15e-76    42.3
```

```
# STEP 5 — Filter Hits by Percent Identity
# Accepts all hits with PID <= 100% (PIDcut = 1.00)
# Prints accession, sequence length, alignment length, E-value, PID%, and coverage%

PIDcut = 1.00

def filter_hits(blast_record, query_len, label):
    print(f"\nFiltered hits: {label}")
    print(f"{'Accession':<28} {'SeqLen':>7} {'AlnLen':>7} {'E-val':>10} {'PID%':>6} {'Cov%':>6}")
    print("-" * 70)
    passed = []
    for aln in blast_record.alignments:
        for hsp in aln.hsps:
            pid = hsp.identities / hsp.align_length
            if pid <= PIDcut:
                cov = 100 * sum(c.isalpha() for c in hsp.query) / query_len
                print(f"{aln.hit_id:<28} {aln.length:>7} {hsp.align_length:>7} "
                      f"{hsp.expect:>10.2e} {100*pid:>6.1f} {cov:>6.1f}")
                passed.append(aln.hit_id)
    print(f"  {len(passed)} hits passed filter")
    return passed

kinesin_hits   = filter_hits(blast_kinesin, len(query_records["KIF5B_Hs_Kinesin1"].seq),  "Kinesin motor")
kinesin14_hits = filter_hits(blast_k14,     len(query_records["KIFC1_Hs_Kinesin14"].seq), "Kinesin-14")
```

```
Filtered hits: Kinesin motor
Accession                    SeqLen  AlnLen      E-val   PID%   Cov%
----------------------------------------------------------------------
pdb|8RHB|K                      963     963   0.00e+00  100.0  100.0
pdb|9GNQ|K                      408     357   0.00e+00  100.0   37.1
pdb|9GNQ|K                      408      24   1.14e-02   70.8    2.5
pdb|1MKJ|A                      349     349   0.00e+00  100.0   36.2
pdb|8IXA|S                      372     349   0.00e+00   99.7   36.2
pdb|9L7M|K                      357     349   0.00e+00   99.4   36.2
pdb|3J8X|K                      349     349   0.00e+00   98.9   36.2
pdb|4HNA|K                      349     349   0.00e+00   98.3   36.2
pdb|9L7E|A                      355     349   0.00e+00   98.0   36.2
pdb|4ATX|C                      340     340   0.00e+00   98.8   35.3
pdb|1BG2|A                      325     325   0.00e+00  100.0   33.7
pdb|9L78|A                      342     335   0.00e+00   97.9   34.8
pdb|9L6K|A                      342     335   0.00e+00   97.9   34.8
pdb|4LNU|K                      325     325   0.00e+00   98.5   33.7
pdb|5LT3|A                      325     325   0.00e+00   98.2   33.7
pdb|5LT1|A                      325     325   0.00e+00   98.2   33.7
pdb|6OJQ|K                      317     317   0.00e+00   98.7   32.9
pdb|3J6H|K                      352     342   0.00e+00   86.8   35.3
pdb|3WRD|A                      341     334   0.00e+00   87.1   34.5
pdb|4UXT|C                      340     334   0.00e+00   85.3   34.6
pdb|5HNY|K                      371     321   0.00e+00   87.2   33.3
pdb|5HLE|A                      371     321   0.00e+00   85.7   33.3
pdb|2Y5W|A                      365     355   0.00e+00   75.8   36.4
pdb|2KIN|A                      238     238  3.86e-148   85.3   24.6
pdb|1GOJ|A                      355     347  8.08e-126   55.6   35.4
pdb|1IA0|K                      394     372   3.90e-84   42.5   35.1
pdb|3B6U|A                      372     342   2.60e-80   45.3   33.7
pdb|1I5S|A                      367     363   3.70e-79   42.1   34.2
pdb|1VFV|A                      366     363   4.69e-79   42.1   34.2
pdb|1I6I|A                      366     363   4.93e-79   42.1   34.2
pdb|7A3Z|A                      372     343   8.63e-79   45.2   34.2
  31 hits passed filter

Filtered hits: Kinesin-14
Accession                    SeqLen  AlnLen      E-val   PID%   Cov%
----------------------------------------------------------------------
pdb|5WDH|A                      376     357   0.00e+00   99.7   53.0
pdb|2NCD|A                      420     433   5.49e-97   41.8   63.3
pdb|4GKR|A                      371     378   1.18e-95   44.7   54.4
pdb|5W3D|A                      412     421   1.91e-95   42.5   61.5
pdb|1CZ7|A                      406     413   7.21e-95   42.4   60.3
pdb|3U06|A                      412     421   1.54e-94   42.3   61.5
pdb|1N6M|A                      409     421   1.81e-94   42.3   61.5
pdb|8YUE|A                      390     421   3.55e-94   42.3   61.5
pdb|8YY2|C                      409     421   5.49e-94   42.3   61.5
pdb|4ETP|A                      403     412   6.35e-94   41.5   59.7
pdb|1F9T|A                      358     379   7.30e-94   44.6   54.2
```

```
pdb|3L1C|A                    383     412   1.92e-93   42.5   60.2
pdb|3T0Q|A                    349     371   1.23e-90   45.8   53.5
pdb|3KAR|A                    346     367   2.16e-86   43.6   52.5
pdb|1F9W|A                    347     369   5.62e-86   43.1   52.7
pdb|1F9U|A                    347     369   6.52e-86   43.1   52.7
pdb|1F9V|A                    347     369   7.57e-86   43.1   52.7
```

```
# STEP 6 — Download Top Hits from Both Searches and Merge into One FASTA
# Takes top 15 hits from each BLAST search (30 total)
# Deduplicates by sequence, adds both query sequences as anchors at the top
# Saves combined output to data/sequences.fasta

from Bio import SeqIO, Entrez
import time

Entrez.email = "your.email@example.com"   # <-- Replace with your email

def fetch_hits(blast_record, top_n, label, pid_cut=1.00):
    records = []
    seen_ids = set()
    count = 0
    for aln in blast_record.alignments[:top_n]:
        for hsp in aln.hsps:
            if aln.hit_id in seen_ids:
                continue
            pid = hsp.identities / len(hsp.match)
            if pid <= pid_cut:
                print(f"  Fetching [{label}]: {aln.hit_id} ... ", end="")
                try:
                    handle = Entrez.efetch(db="protein", id=aln.hit_id, rettype="fasta")
                    rec = SeqIO.read(handle, "fasta")
                    handle.close()
                    records.append(rec)
                    seen_ids.add(aln.hit_id)
                    count += 1
                    print("OK")
                except Exception as e:
                    print(f"FAILED ({e})")
                time.sleep(0.4)
    print(f"  Fetched {count} sequences for {label}")
    return records

print("Fetching Kinesin motor hits...")
kinesin_seqs = fetch_hits(blast_kinesin, top_n=15, label="KinesinMotor")

print("\nFetching Kinesin-14 hits...")
kinesin14_seqs = fetch_hits(blast_k14, top_n=15, label="Kinesin14")

# Merge: query sequences first, then BLAST hits, deduplicate by sequence string
all_records = list(query_records.values())
seen_seqs = {str(r.seq).upper() for r in all_records}

for rec in kinesin_seqs + kinesin14_seqs:
    seq_str = str(rec.seq).upper()
    if seq_str not in seen_seqs:
        all_records.append(rec)
        seen_seqs.add(seq_str)

print(f"\nTotal unique sequences (both groups + queries): {len(all_records)}")

SeqIO.write(all_records, "data/sequences.fasta", "fasta")
print("Saved → data/sequences.fasta")
```

```
Fetching Kinesin motor hits...
  Fetching [KinesinMotor]: pdb|8RHB|K ... OK
  Fetching [KinesinMotor]: pdb|9GNQ|K ... OK
  Fetching [KinesinMotor]: pdb|1MKJ|A ... OK
  Fetching [KinesinMotor]: pdb|8IXA|S ... OK
  Fetching [KinesinMotor]: pdb|9L7M|K ... OK
  Fetching [KinesinMotor]: pdb|3J8X|K ... OK
  Fetching [KinesinMotor]: pdb|4HNA|K ... OK
  Fetching [KinesinMotor]: pdb|9L7E|A ... OK
  Fetching [KinesinMotor]: pdb|4ATX|C ... OK
  Fetching [KinesinMotor]: pdb|1BG2|A ... OK
  Fetching [KinesinMotor]: pdb|9L78|A ... OK
  Fetching [KinesinMotor]: pdb|9L6K|A ... OK
  Fetching [KinesinMotor]: pdb|4LNU|K ... OK
  Fetching [KinesinMotor]: pdb|5LT3|A ... OK
  Fetching [KinesinMotor]: pdb|5LT1|A ... OK
```

```
  Fetched 15 sequences for KinesinMotor

Fetching Kinesin-14 hits...
  Fetching [Kinesin14]: pdb|5WDH|A ... OK
  Fetching [Kinesin14]: pdb|2NCD|A ... OK
  Fetching [Kinesin14]: pdb|4GKR|A ... OK
  Fetching [Kinesin14]: pdb|5W3D|A ... OK
  Fetching [Kinesin14]: pdb|1CZ7|A ... OK
  Fetching [Kinesin14]: pdb|3U06|A ... OK
  Fetching [Kinesin14]: pdb|1N6M|A ... OK
  Fetching [Kinesin14]: pdb|8YUE|A ... OK
  Fetching [Kinesin14]: pdb|8YY2|C ... OK
  Fetching [Kinesin14]: pdb|4ETP|A ... OK
  Fetching [Kinesin14]: pdb|1F9T|A ... OK
  Fetching [Kinesin14]: pdb|3L1C|A ... OK
  Fetching [Kinesin14]: pdb|3T0Q|A ... OK
  Fetching [Kinesin14]: pdb|3KAR|A ... OK
  Fetching [Kinesin14]: pdb|1F9W|A ... OK
  Fetched 15 sequences for Kinesin14

Total unique sequences (both groups + queries): 31
Saved → data/sequences.fasta
```

```
# STEP 7 — Multiple Sequence Alignment with MAFFT
# Uses auto-detection strategy to align all sequences
# Output saved to data/aligned.fasta

!mafft --auto data/sequences.fasta > data/aligned.fasta
```

```
outputhat23=16
treein = 0
compacttree = 0
stacksize: 8192 kb
rescale = 1
All-to-all alignment.
tbfast-pair (aa) Version 7.490
alg=L, model=BLOSUM62, 2.00, -0.10, +0.10, noshift, amax=0.0
0 thread(s)

outputhat23=16
Loading 'hat3.seed' ...
done.
Writing hat3 for iterative refinement
rescale = 1
Gap Penalty = -1.53, +0.00, +0.00
tbutree = 1, compacttree = 0
Constructing a UPGMA tree ...
   20 / 31
done.

Progressive alignment ...
STEP    30 /30
done.
tbfast (aa) Version 7.490
alg=A, model=BLOSUM62, 1.53, -0.00, -0.00, noshift, amax=0.0
1 thread(s)

minimumweight = 0.000010
autosubalignment = 0.000000
nthread = 0
randomseed = 0
blosum 62 / kimura 200
poffset = 0
niter = 16
sueff_global = 0.100000
nadd = 16
Loading 'hat3' ... done.
rescale = 1

   20 / 31
Segment   1/  1    1-1325
STEP 003-015-0  identical.
Converged.

done
dvtditr (aa) Version 7.490
alg=A, model=BLOSUM62, 1.53, -0.00, -0.00, noshift, amax=0.0
0 thread(s)


Strategy:
 L-INS-i (Probably most accurate, very slow)
 Iterative refinement method (<16) with LOCAL pairwise alignment information
```

```
If unsure which option to use, try 'mafft --auto input > output'.
For more information, see 'mafft --help', 'mafft --man' and the mafft page.
```

```
# Verify alignment dimensions

from Bio import AlignIO
aln = AlignIO.read("data/aligned.fasta", "fasta")
print(f"Alignment: {len(aln)} sequences × {aln.get_alignment_length()} columns")
```

```
Alignment: 31 sequences × 1324 columns
```

```
# STEP 8 — Trim Alignment to Remove Gappy Terminal Columns
# Scans from the left to find the first fully ungapped column (position1)
# Scans from the right to find the last fully ungapped column (position2)
# Slices the alignment to that range and saves as data/aligned_trimmed.fasta

from Bio import AlignIO, SeqIO

aln = AlignIO.read("data/aligned.fasta", "fasta")

position1 = 0
for col in range(aln.get_alignment_length()):
    if "-" not in aln[:, col]:
        position1 = col
        break

position2 = aln.get_alignment_length()
for col in reversed(range(aln.get_alignment_length())):
    if "-" not in aln[:, col]:
        position2 = col + 1
        break

print(f"Original length : {aln.get_alignment_length()} columns")
print(f"Keeping columns : {position1} → {position2}")
print(f"Trimmed length  : {position2 - position1} columns")

with open("data/aligned_trimmed.fasta", "w") as fh:
    SeqIO.write(aln[:, position1:position2], fh, "fasta")

trim = AlignIO.read("data/aligned_trimmed.fasta", "fasta")
print(f"Trimmed alignment: {len(trim)} sequences × {trim.get_alignment_length()} columns")
```

```
Original length : 1324 columns
Keeping columns : 314 → 685
Trimmed length  : 371 columns
Trimmed alignment: 31 sequences × 371 columns
```

```
# STEP 9 — Compute Pairwise Distance Matrix
# Uses the identity model via BioPython's DistanceCalculator
# Gives a numerical overview of sequence similarity before tree building

from Bio.Phylo.TreeConstruction import DistanceCalculator

calculator = DistanceCalculator("identity")
dm = calculator.get_distance(trim)

print("Pairwise distance matrix (identity model):\n")
print(dm)
```

```
Pairwise distance matrix (identity model):

KIF5B_Hs_Kinesin1   0.000000
KIFC1_Hs_Kinesin14  0.654987    0.000000
pdb|9GNQ|K  0.000000    0.654987    0.000000
pdb|1MKJ|A  0.000000    0.654987    0.000000    0.000000
pdb|8IXA|S  0.002695    0.657682    0.002695    0.002695    0.000000
pdb|9L7M|K  0.000000    0.654987    0.000000    0.000000    0.002695    0.000000
pdb|3J8X|K  0.008086    0.649596    0.008086    0.008086    0.010782    0.008086    0.000000
pdb|4HNA|K  0.013477    0.646900    0.013477    0.013477    0.016173    0.013477    0.005391    0.000000
pdb|9L7E|A  0.016173    0.649596    0.016173    0.016173    0.018868    0.016173    0.008086    0.002695    0.000000
pdb|4ATX|C  0.008086    0.657682    0.008086    0.008086    0.010782    0.008086    0.016173    0.021563    0.024259    0.000000
pdb|1BG2|A  0.000000    0.654987    0.000000    0.000000    0.002695    0.000000    0.008086    0.013477    0.016173    0.008086
pdb|9L78|A  0.016173    0.649596    0.016173    0.016173    0.018868    0.016173    0.008086    0.002695    0.000000    0.024259
pdb|9L6K|A  0.016173    0.649596    0.016173    0.016173    0.018868    0.016173    0.008086    0.002695    0.002695    0.024259
pdb|4LNU|K  0.013477    0.646900    0.013477    0.013477    0.016173    0.013477    0.005391    0.000000    0.002695    0.021563
pdb|5LT3|A  0.016173    0.649596    0.016173    0.016173    0.018868    0.016173    0.008086    0.002695    0.005391    0.024259
pdb|5LT1|A  0.016173    0.649596    0.016173    0.016173    0.018868    0.016173    0.008086    0.002695    0.005391    0.021563
```

```
pdb|5WDH|A  0.654987    0.002695    0.654987    0.654987    0.657682    0.654987    0.649596    0.646900    0.649596    0.657682
pdb|2NCD|A  0.582210    0.563342    0.582210    0.582210    0.584906    0.582210    0.582210    0.582210    0.584906    0.584906
pdb|4GKR|A  0.638814    0.568733    0.638814    0.638814    0.641509    0.638814    0.638814    0.638814    0.641509    0.641509
pdb|5W3D|A  0.582210    0.563342    0.582210    0.582210    0.584906    0.582210    0.582210    0.582210    0.584906    0.584906
pdb|1CZ7|A  0.582210    0.563342    0.582210    0.582210    0.584906    0.582210    0.582210    0.582210    0.584906    0.584906
pdb|3U06|A  0.582210    0.566038    0.582210    0.582210    0.584906    0.582210    0.582210    0.582210    0.584906    0.584906
pdb|1N6M|A  0.584906    0.566038    0.584906    0.584906    0.587601    0.584906    0.584906    0.584906    0.587601    0.587601
pdb|8YUE|A  0.584906    0.566038    0.584906    0.584906    0.587601    0.584906    0.584906    0.584906    0.587601    0.587601
pdb|8YY2|C  0.584906    0.566038    0.584906    0.584906    0.587601    0.584906    0.584906    0.584906    0.587601    0.587601
pdb|4ETP|A  0.628032    0.571429    0.628032    0.628032    0.630728    0.628032    0.628032    0.628032    0.630728    0.630728
pdb|1F9T|A  0.628032    0.571429    0.628032    0.628032    0.630728    0.628032    0.628032    0.628032    0.630728    0.630728
pdb|3L1C|A  0.584906    0.566038    0.584906    0.584906    0.587601    0.584906    0.584906    0.584906    0.587601    0.587601
pdb|3T0Q|A  0.633423    0.560647    0.633423    0.633423    0.636119    0.633423    0.633423    0.633423    0.636119    0.636119
pdb|3KAR|A  0.628032    0.571429    0.628032    0.628032    0.630728    0.628032    0.628032    0.628032    0.630728    0.630728
pdb|1F9W|A  0.630728    0.574124    0.630728    0.630728    0.628032    0.630728    0.630728    0.630728    0.633423    0.633423
    KIF5B_Hs_Kinesin1   KIFC1_Hs_Kinesin14  pdb|9GNQ|K  pdb|1MKJ|A  pdb|8IXA|S  pdb|9L7M|K  pdb|3J8X|K  pdb|4HNA|K  pdb|9L7E|A
```

```
# STEP 10 — Remove Duplicate Sequences
# Reads the trimmed alignment and removes exact duplicate sequences
# When duplicates are found their IDs are merged with an underscore
# Cleaned output saved as data/clear_aligned_trimmed.fasta

from Bio import SeqIO

def sequence_cleaner(fasta_file, min_length=0):
    sequences = {}
    for rec in SeqIO.parse(fasta_file, "fasta"):
        seq = str(rec.seq).upper()
        if len(seq) >= min_length:
            if seq not in sequences:
                sequences[seq] = rec.id
            else:
                sequences[seq] += "_" + rec.id

    out_file = "data/clear_" + os.path.basename(fasta_file)
    with open(out_file, "w") as fh:
        for seq, seq_id in sequences.items():
            fh.write(">" + seq_id + "\n" + seq + "\n")

    n_in  = sum(1 for _ in SeqIO.parse(fasta_file, "fasta"))
    n_out = len(sequences)
    print(f"Input:  {n_in} sequences")
    print(f"Output: {n_out} sequences (removed {n_in - n_out} duplicates)")
    print(f"Saved → {out_file}")
    return out_file

cleaned_file = sequence_cleaner("data/aligned_trimmed.fasta", min_length=0)
```

```
Input:  31 sequences
Output: 22 sequences (removed 9 duplicates)
Saved → data/clear_aligned_trimmed.fasta
```

```
# STEP 11 — Model Selection with ModelTest-NG
# Evaluates all candidate amino acid substitution models on the cleaned alignment
# Best model selected based on the Bayesian Information Criterion (BIC)
# Typical result for kinesins: LG+I+G4 or WAG+G4

!modeltest-ng -i {cleaned_file} -d aa
```

```
/bin/bash: line 1: modeltest-ng: command not found
```

```
# Update MODEL below based on the BIC result printed above

MODEL = "LG+I+G4"   # <-- Update if ModelTest-NG recommends a different model
print(f"Selected substitution model: {MODEL}")
```

```
Selected substitution model: LG+I+G4
```

```
# STEP 12 — Build Maximum-Likelihood Tree with RAxML-NG
# Infers the best-scoring ML tree under the selected substitution model
# Results prefixed with T1, producing T1.raxml.bestTree and T1.raxml.rba

!rm -f T1.raxml.*

!raxml-ng \
    --msa {cleaned_file} \
```

```
    --model {MODEL} \
    --prefix T1 \
    --threads 2 \
    --seed 12345
```

```
RAxML-NG v. 1.2.0 released on 09.05.2023 by The Exelixis Lab.
Developed by: Alexey M. Kozlov and Alexandros Stamatakis.
Contributors: Diego Darriba, Tomas Flouri, Benoit Morel, Sarah Lutteropp, Ben Bettisworth, Julia Haag, Anastasis Togkousidis.
Latest version: https://github.com/amkozlov/raxml-ng
Questions/problems/suggestions? Please visit: https://groups.google.com/forum/#!forum/raxml

System: Intel(R) Xeon(R) CPU @ 2.20GHz, 1 cores, 12 GB RAM

RAxML-NG was called at 25-Apr-2026 13:32:01 as follows:

raxml-ng --msa data/clear_aligned_trimmed.fasta --model LG+I+G4 --prefix T1 --threads 2 --seed 12345

Analysis options:
  run mode: ML tree search
  start tree(s): random (10) + parsimony (10)
  random seed: 12345
  tip-inner: OFF
  pattern compression: ON
  per-rate scalers: OFF
  site repeats: ON
  logLH epsilon: general: 10.000000, brlen-triplet: 1000.000000
  fast spr radius: AUTO
  spr subtree cutoff: 1.000000
  fast CLV updates: ON
  branch lengths: proportional (ML estimate, algorithm: NR-FAST)
  SIMD kernels: AVX2
  parallelization: coarse-grained (auto), PTHREADS (2 threads), thread pinning: OFF

[00:00:00] Reading alignment from file: data/clear_aligned_trimmed.fasta
[00:00:00] Loaded alignment with 22 taxa and 371 sites

Alignment comprises 1 partitions and 305 patterns

Partition 0: noname
Model: LG+I+G4m
Alignment sites / patterns: 371 / 305
Gaps: 11.09 %
Invariant sites: 26.42 %


NOTE: Binary MSA file created: T1.raxml.rba

Parallelization scheme autoconfig: 2 worker(s) x 1 thread(s)

[00:00:00] Generating 10 random starting tree(s) with 22 taxa
[00:00:00] Generating 10 parsimony starting tree(s) with 22 taxa
Parallel parsimony with 2 threads
Parallel reduction/worker buffer size: 1 KB  / 0 KB

[00:00:00] Data distribution: max. partitions/sites/weight per thread: 1 / 305 / 24400
[00:00:00] Data distribution: max. searches per worker: 10

Starting ML tree search with 20 distinct starting trees

[00:00:00 -11612.312388] Initial branch length optimization
[00:00:00 -8334.138156] Model parameter optimization (eps = 10.000000)
```

```
# Verify that the best tree file was produced

if os.path.exists("T1.raxml.bestTree"):
    print("Best ML tree found: T1.raxml.bestTree")
    with open("T1.raxml.bestTree") as f:
        print(f.read()[:400])
else:
    print("Tree not found — check RAxML-NG output above")
```

```
Best ML tree found: T1.raxml.bestTree
((pdb|3T0Q|A:0.332906,(pdb|4GKR|A:0.220990,(pdb|4ETP|A:0.013896,((pdb|3KAR|A:0.002764,pdb|1F9T|A:0.000001):0.000001,pdb|1F9W|A:0
```

```
# STEP 13 — Visualize Initial ML Tree
# Reads T1.raxml.bestTree and renders it as a matplotlib figure
# Query sequences highlighted: KIF5B in blue, KIFC1 in red

from Bio import Phylo
```

```
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches

tree = Phylo.read("T1.raxml.bestTree", "newick")
tree.ladderize()

COLORS = {
    "KIF5B_Hs_Kinesin1":  "#2196F3",
    "KIFC1_Hs_Kinesin14": "#E53935",
}

def leaf_label_func(clade):
    return clade.name if clade.name else ""

fig, ax = plt.subplots(figsize=(14, max(8, len(list(tree.get_terminals())) * 0.35)))
Phylo.draw(tree, axes=ax, do_show=False, label_func=leaf_label_func)

for text in ax.texts:
    if text.get_text() in COLORS:
        text.set_color(COLORS[text.get_text()])
        text.set_fontweight("bold")

ax.set_title(
    "Kinesin Motor + Kinesin-14 Combined — Best ML Tree (no bootstrap)\n"
    f"Model: {MODEL}  |  Blue = KIF5B (Kinesin-1)  |  Red = KIFC1 (Kinesin-14)",
    fontsize=12, pad=12
)
legend_handles = [
    mpatches.Patch(color="#2196F3", label="Kinesin motor (Kinesin-1, KIF5B) hits"),
    mpatches.Patch(color="#E53935", label="Kinesin-14 (KIFC1/HSET) hits"),
]
ax.legend(handles=legend_handles, loc="lower right", fontsize=9)
plt.tight_layout()
plt.savefig("data/tree_ML.png", dpi=150, bbox_inches="tight")
plt.show()
print("Saved → data/tree_ML.png")
```

```
Saved → data/tree_ML.png
```

```
# STEP 14 — Bootstrap Analysis (100 Replicates)
# Uses the binary alignment file T1.raxml.rba produced in Step 12
# Bootstrap trees saved with prefix T2
# NOTE: This may take several minutes

!rm -f T2.raxml.*

!raxml-ng \
    --bootstrap \
    --msa T1.raxml.rba \
    --model {MODEL} \
    --prefix T2 \
    --threads 2 \
    --seed 12345 \
    --bs-trees 100

print("Bootstrap trees → T2.raxml.bootstraps")
```

```
RAxML-NG v. 1.2.0 released on 09.05.2023 by The Exelixis Lab.
Developed by: Alexey M. Kozlov and Alexandros Stamatakis.
Contributors: Diego Darriba, Tomas Flouri, Benoit Morel, Sarah Lutteropp, Ben Bettisworth, Julia Haag, Anastasis Togkousidis.
Latest version: https://github.com/amkozlov/raxml-ng
Questions/problems/suggestions? Please visit: https://groups.google.com/forum/#!forum/raxml

System: Intel(R) Xeon(R) CPU @ 2.20GHz, 1 cores, 12 GB RAM

RAxML-NG was called at 25-Apr-2026 13:34:11 as follows:

raxml-ng --bootstrap --msa T1.raxml.rba --model LG+I+G4 --prefix T2 --threads 2 --seed 12345 --bs-trees 100

Analysis options:
  run mode: Bootstrapping
  start tree(s):
  bootstrap replicates: parsimony (100)
  random seed: 12345
  tip-inner: OFF
```

```
  pattern compression: ON
  per-rate scalers: OFF
  site repeats: ON
  logLH epsilon: general: 10.000000, brlen-triplet: 1000.000000
  fast spr radius: AUTO
  spr subtree cutoff: 1.000000
  fast CLV updates: ON
  branch lengths: proportional (ML estimate, algorithm: NR-FAST)
  SIMD kernels: AVX2
  parallelization: coarse-grained (auto), PTHREADS (2 threads), thread pinning: OFF

WARNING: The model you specified on the command line (LG+I+G4) will be ignored
         since the binary MSA file already contains a model definition.
         If you want to change the model, please re-run RAxML-NG
         with the original PHYLIP/FASTA alignment and --redo option.

[00:00:00] Loading binary alignment from file: T1.raxml.rba
[00:00:00] Alignment comprises 22 taxa, 1 partitions and 305 patterns

Partition 0: noname
Model: LG+I+G4m
Alignment sites / patterns: 371 / 305
Gaps: 11.09 %
Invariant sites: 26.42 %


Parallelization scheme autoconfig: 2 worker(s) x 1 thread(s)

Parallel reduction/worker buffer size: 1 KB  / 0 KB

[00:00:00] Data distribution: max. partitions/sites/weight per thread: 1 / 305 / 24400
[00:00:00] Data distribution: max. searches per worker: 50
[00:00:00] Starting bootstrapping analysis with 100 replicates.

[00:00:05] [worker #1] Bootstrap tree #2, logLikelihood: -4020.037403
[00:00:06] [worker #0] Bootstrap tree #1, logLikelihood: -4164.684315
[00:00:10] [worker #1] Bootstrap tree #4, logLikelihood: -4021.612124
[00:00:11] [worker #0] Bootstrap tree #3, logLikelihood: -4142.856980
```

```
# STEP 15 — Map Bootstrap Support Values onto Best ML Tree
# Maps the 100 bootstrap replicate trees onto the best ML tree
# Support values at each node indicate confidence in the branching pattern
# Annotated tree saved with prefix T3

!rm -f T3.raxml.*

!raxml-ng \
    --support \
    --tree T1.raxml.bestTree \
    --bs-trees T2.raxml.bootstraps \
    --prefix T3 \
    --threads 2

print("Bootstrap-annotated tree → T3.raxml.support")
```

```
RAxML-NG v. 1.2.0 released on 09.05.2023 by The Exelixis Lab.
Developed by: Alexey M. Kozlov and Alexandros Stamatakis.
Contributors: Diego Darriba, Tomas Flouri, Benoit Morel, Sarah Lutteropp, Ben Bettisworth, Julia Haag, Anastasis Togkousidis.
Latest version: https://github.com/amkozlov/raxml-ng
Questions/problems/suggestions? Please visit: https://groups.google.com/forum/#!forum/raxml

System: Intel(R) Xeon(R) CPU @ 2.20GHz, 1 cores, 12 GB RAM

RAxML-NG was called at 25-Apr-2026 13:39:00 as follows:

raxml-ng --support --tree T1.raxml.bestTree --bs-trees T2.raxml.bootstraps --prefix T3 --threads 2

Analysis options:
  run mode: Compute bipartition support (Felsenstein Bootstrap)
  start tree(s): user
  random seed: 1777124340
  SIMD kernels: AVX2
  parallelization: coarse-grained (auto), PTHREADS (2 threads), thread pinning: OFF

Reading reference tree from file: T1.raxml.bestTree
Reference tree size: 22

Reading bootstrap trees from file: T2.raxml.bootstraps
Loaded 100 trees with 22 taxa.

Best ML tree with Felsenstein bootstrap (FBP) support values saved to: /content/T3.raxml.support
```

```
Execution log saved to: /content/T3.raxml.log

Analysis started: 25-Apr-2026 13:39:00 / finished: 25-Apr-2026 13:39:00

Elapsed time: 0.006 seconds

Bootstrap-annotated tree → T3.raxml.support
```

```
# STEP 16 — Visualize Bootstrap Support Tree
# Reads T3.raxml.support and renders it with bootstrap values at nodes
# Query sequences highlighted: KIF5B in blue, KIFC1 in red

from Bio import Phylo
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches

tree = Phylo.read("T3.raxml.support", "newick")
tree.ladderize()

fig, ax = plt.subplots(figsize=(14, max(8, len(list(tree.get_terminals())) * 0.35)))
Phylo.draw(tree, axes=ax, do_show=False)

for text in ax.texts:
    if "KIF5B_Hs_Kinesin1" in text.get_text():
        text.set_color("#2196F3"); text.set_fontweight("bold")
    elif "KIFC1_Hs_Kinesin14" in text.get_text():
        text.set_color("#E53935"); text.set_fontweight("bold")

ax.set_title(
    "Kinesin Motor + Kinesin-14 Combined — ML Tree with 100 Bootstrap Support\n"
    f"Model: {MODEL}  |  Node labels = bootstrap % (100 replicates)",
    fontsize=12, pad=12
)
legend_handles = [
    mpatches.Patch(color="#2196F3", label="Kinesin motor (Kinesin-1) query & hits"),
    mpatches.Patch(color="#E53935", label="Kinesin-14 query & hits"),
]
ax.legend(handles=legend_handles, loc="lower right", fontsize=9)
plt.tight_layout()
plt.savefig("data/tree_bootstrap.png", dpi=150, bbox_inches="tight")
plt.show()
print("Saved → data/tree_bootstrap.png")
```

```
Saved → data/tree_bootstrap.png
```

```
# STEP 17 — Ancestral Sequence Reconstruction
# Runs RAxML-NG in ancestral mode to reconstruct marginal ancestral amino acid
# states at every internal node of the best ML tree
# Outputs: ASR.raxml.ancestralTree and ASR.raxml.ancestralProbs

!raxml-ng \
    --ancestral \
    --msa {cleaned_file} \
    --tree T1.raxml.bestTree \
    --model {MODEL} \
    --prefix ASR

for fname in ["ASR.raxml.ancestralTree", "ASR.raxml.ancestralProbs"]:
    status = "Found" if os.path.exists(fname) else "Missing"
    print(f"{status}: {fname}")
```

```
RAxML-NG v. 1.2.0 released on 09.05.2023 by The Exelixis Lab.
Developed by: Alexey M. Kozlov and Alexandros Stamatakis.
Contributors: Diego Darriba, Tomas Flouri, Benoit Morel, Sarah Lutteropp, Ben Bettisworth, Julia Haag, Anastasis Togkousidis.
Latest version: https://github.com/amkozlov/raxml-ng
Questions/problems/suggestions? Please visit: https://groups.google.com/forum/#!forum/raxml

System: Intel(R) Xeon(R) CPU @ 2.20GHz, 1 cores, 12 GB RAM

RAxML-NG was called at 25-Apr-2026 13:39:14 as follows:

raxml-ng --ancestral --msa data/clear_aligned_trimmed.fasta --tree T1.raxml.bestTree --model LG+I+G4 --prefix ASR

Analysis options:
  run mode: Ancestral state reconstruction
  start tree(s): user
```

```
  random seed: 1777124354
  tip-inner: ON
  pattern compression: OFF
  per-rate scalers: OFF
  site repeats: OFF
  logLH epsilon: general: 10.000000, brlen-triplet: 1000.000000
  branch lengths: proportional (ML estimate, algorithm: NR-FAST)
  SIMD kernels: AVX2
  parallelization: coarse-grained (auto), NONE/sequential

[00:00:00] Reading alignment from file: data/clear_aligned_trimmed.fasta
[00:00:00] Loaded alignment with 22 taxa and 371 sites

Alignment comprises 1 partitions and 371 sites

Partition 0: noname
Model: LG+I+G4m
Alignment sites: 371
Gaps: 11.09 %
Invariant sites: 26.42 %


NOTE: Binary MSA file created: ASR.raxml.rba

Parallelization scheme autoconfig: 1 worker(s) x 1 thread(s)

[00:00:00] Loading user starting tree(s) from: T1.raxml.bestTree
Parallel reduction/worker buffer size: 1 KB  / 0 KB

[00:00:00] Data distribution: max. partitions/sites/weight per thread: 1 / 371 / 29680
[00:00:00] Data distribution: max. searches per worker: 1

Starting ML tree search with 1 distinct starting trees

[00:00:00] Tree #1, initial LogLikelihood: -4103.627686

[00:00:00 -4103.627686] Initial branch length optimization
[00:00:00 -4103.495386] Model parameter optimization (eps = 10.000000)

[00:00:00] Tree #1, final logLikelihood: -4101.897191
```

```
# STEP 18 — Visualize Ancestral Reconstruction Tree
# Reads ASR.raxml.ancestralTree and renders it with labelled internal nodes
# Internal node labels reference ASR.raxml.ancestralProbs for per-node amino acid probabilities

from Bio import Phylo
import matplotlib.pyplot as plt

tree = Phylo.read("ASR.raxml.ancestralTree", "newick")
tree.ladderize()

fig, ax = plt.subplots(figsize=(14, max(8, len(list(tree.get_terminals())) * 0.35)))
Phylo.draw(tree, axes=ax, do_show=False)

ax.set_title(
    "Kinesin Motor + Kinesin-14 — Ancestral Reconstruction Tree\n"
    "Internal node labels reference ASR.raxml.ancestralProbs for per-node amino acid probabilities",
    fontsize=12, pad=12
)
plt.tight_layout()
plt.savefig("data/tree_ancestral.png", dpi=150, bbox_inches="tight")
plt.show()
print("Saved → data/tree_ancestral.png")
```

```
Saved → data/tree_ancestral.png
```

```
# Download All Output Files to Local Machine

from google.colab import files

output_files = [
    "data/KIF5B_Hs_Kinesin1.fasta",
    "data/KIFC1_Hs_Kinesin14.fasta",
    "data/sequences.fasta",
    "data/aligned.fasta",
    "data/aligned_trimmed.fasta",
    "data/clear_aligned_trimmed.fasta",
    "T1.raxml.bestTree",
    "T1.raxml.rba",
```

```
    "T2.raxml.bootstraps",
    "T3.raxml.support",
    "ASR.raxml.ancestralTree",
    "ASR.raxml.ancestralProbs",
    "data/tree_ML.png",
    "data/tree_bootstrap.png",
    "data/tree_ancestral.png",
]

for f in output_files:
    if os.path.exists(f):
        files.download(f)
        print(f"Downloaded: {f}")
    else:
        print(f"Not found:  {f}")
```

```
Downloaded: data/KIF5B_Hs_Kinesin1.fasta
Downloaded: data/KIFC1_Hs_Kinesin14.fasta
Downloaded: data/sequences.fasta
Downloaded: data/aligned.fasta
Downloaded: data/aligned_trimmed.fasta
Downloaded: data/clear_aligned_trimmed.fasta
Downloaded: T1.raxml.bestTree
Downloaded: T1.raxml.rba
Downloaded: T2.raxml.bootstraps
Downloaded: T3.raxml.support
Downloaded: ASR.raxml.ancestralTree
Downloaded: ASR.raxml.ancestralProbs
Downloaded: data/tree_ML.png
Downloaded: data/tree_bootstrap.png
Downloaded: data/tree_ancestral.png
```